GRADUATION THESIS

Using Lightweight Graph Convolutional Neural Network in Recommender System

Quang Dung Nguyen [1*], Quoc Lap Dinh [1†], Ba Hoang Nam Nguyen [1‡]

FPT University, Education Zone, Hoa Lac Hi-tech Park, Ha Noi
*dungnqhe153335@fpt.edu.vn, †lapdqhe161860@fpt.edu.vn, ‡namnbhhe161029@fpt.edu.vn
*0009-0005-9898-2755, †0009-0008-8472-9654, ‡0009-0009-5677-4252
Advisor: Ph.D. Van Hieu Bui

December 19, 2024

Outline

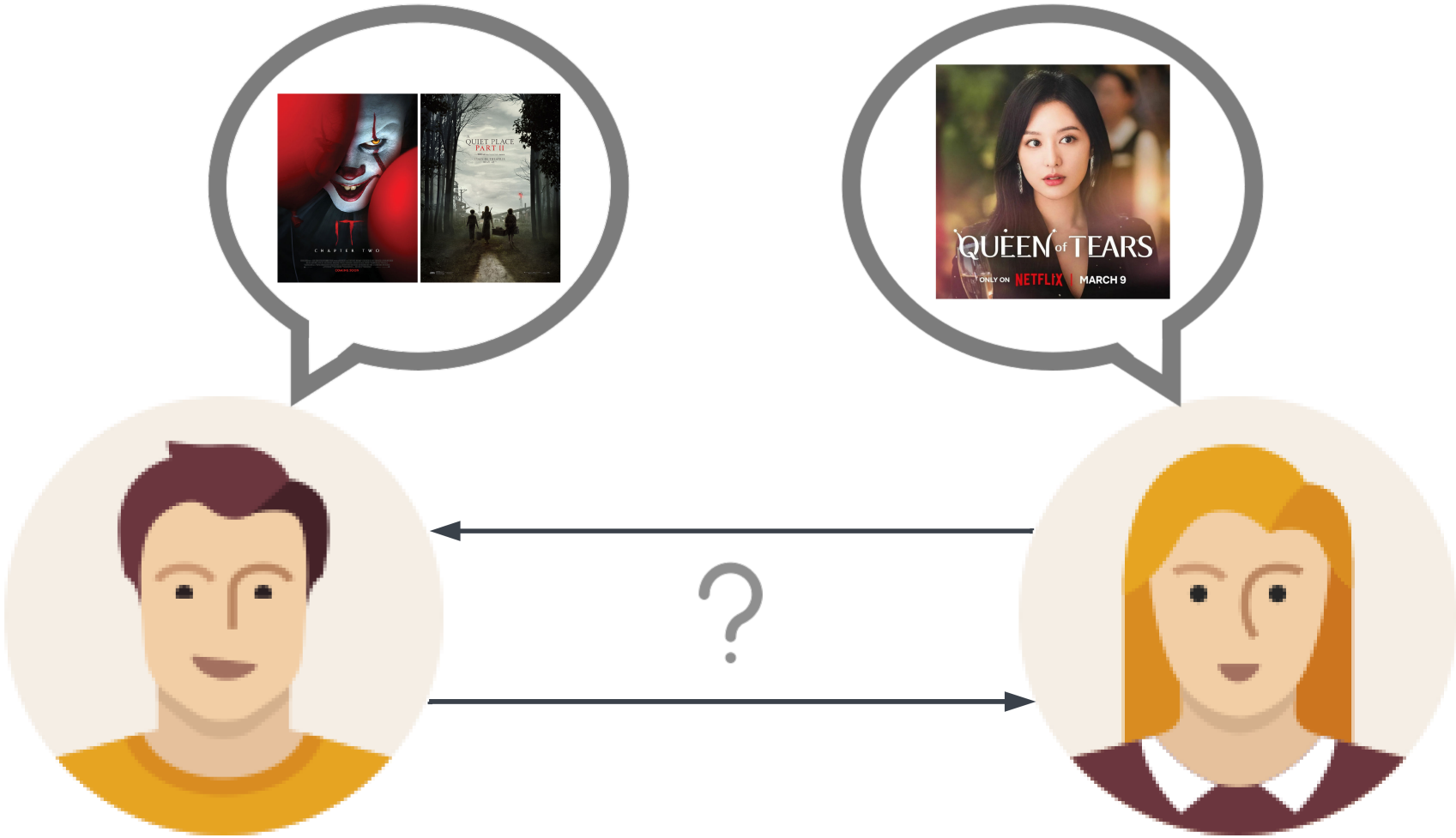
- Introduction
  - Motivation
  - What is Recommendation System (RecSys)?
  - Baseline Models for RecSys
  - Problem Statement
  - Objective
- Related Work
  - Graph Theory
  - Graph Transformations
  - Graph Embedding
  - Graph Convolution
- Methodology
  - Preprocessing: Constructing Efficient Heterogeneous Graphs
  - Supernode Embedding
  - Refinement
  - Prediction and Recommendation

Outline (cont.)

- Evaluation Metrics
- Experiment and Results
- Conclusions
  - Key Contributions
  - Limitations
  - Future Work

Introduction

Motivation



Introduction

What is Recommendation System (RecSys)?

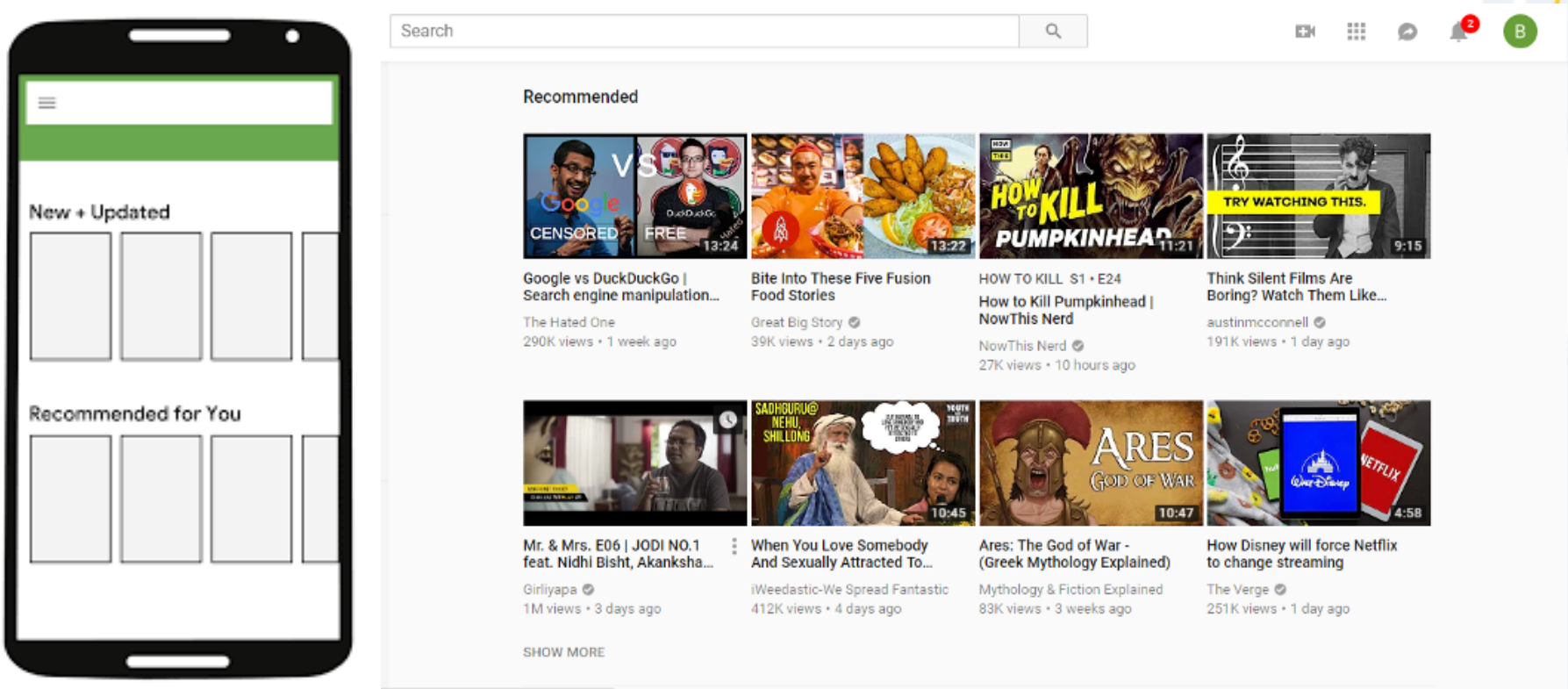


Figure 1 – Examples of RecSys of real-world applications

Introduction

Baseline Models for RecSys

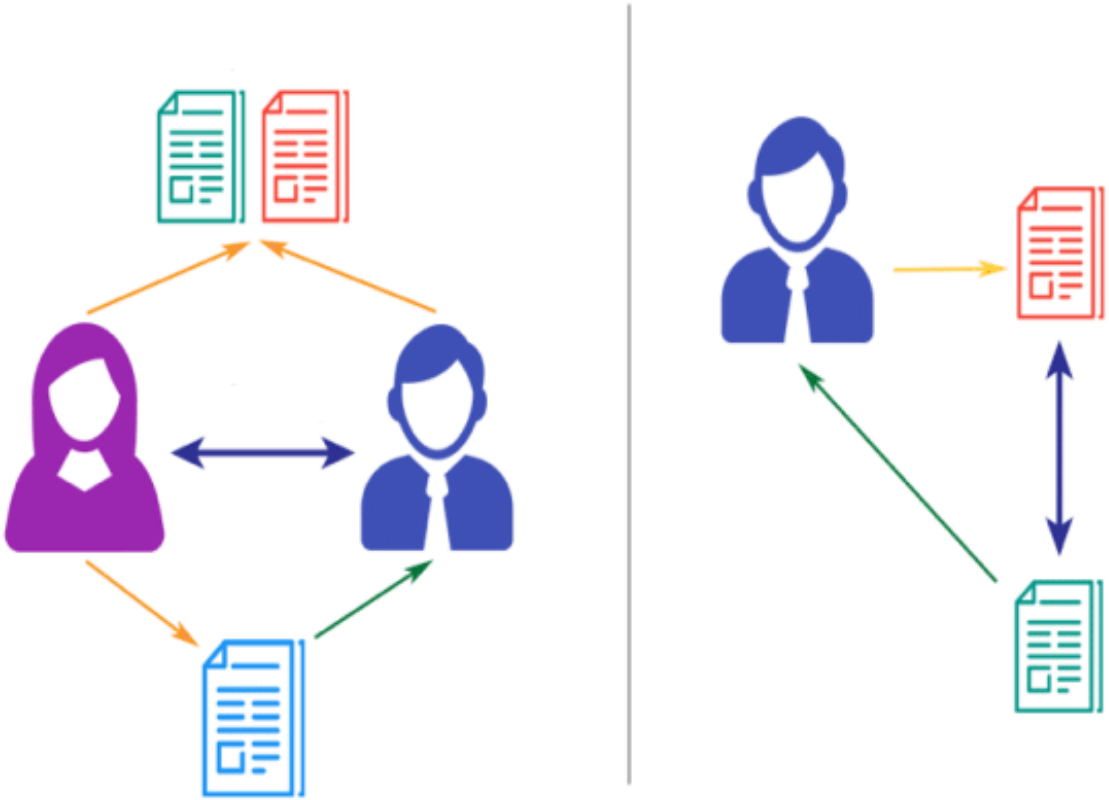


Figure 2 – Collaborative filtering vs. content-based filtering

Introduction
Problem Statement

- The popular problem of Baseline Models

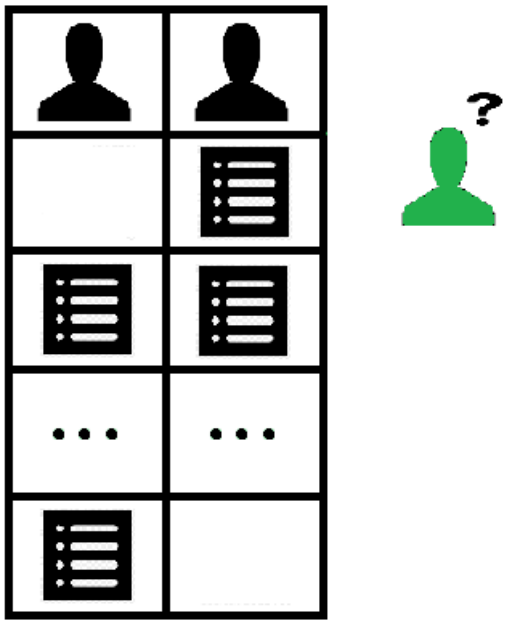


Figure 3 – Cold-start problem

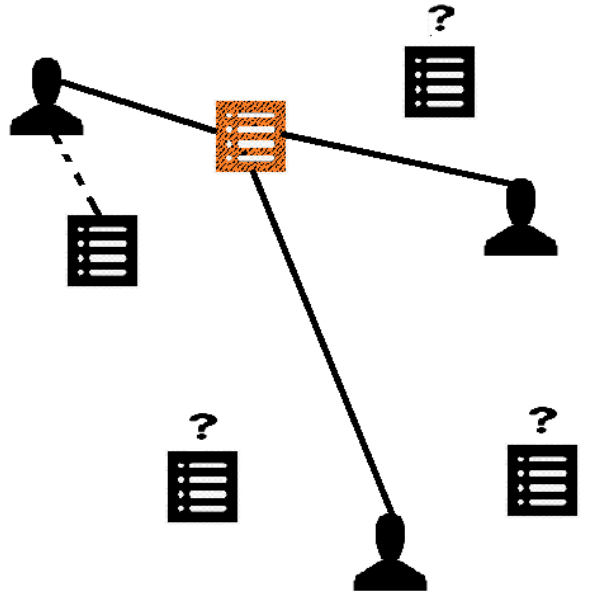


Figure 4 – Long tail problem

Introduction

Objective

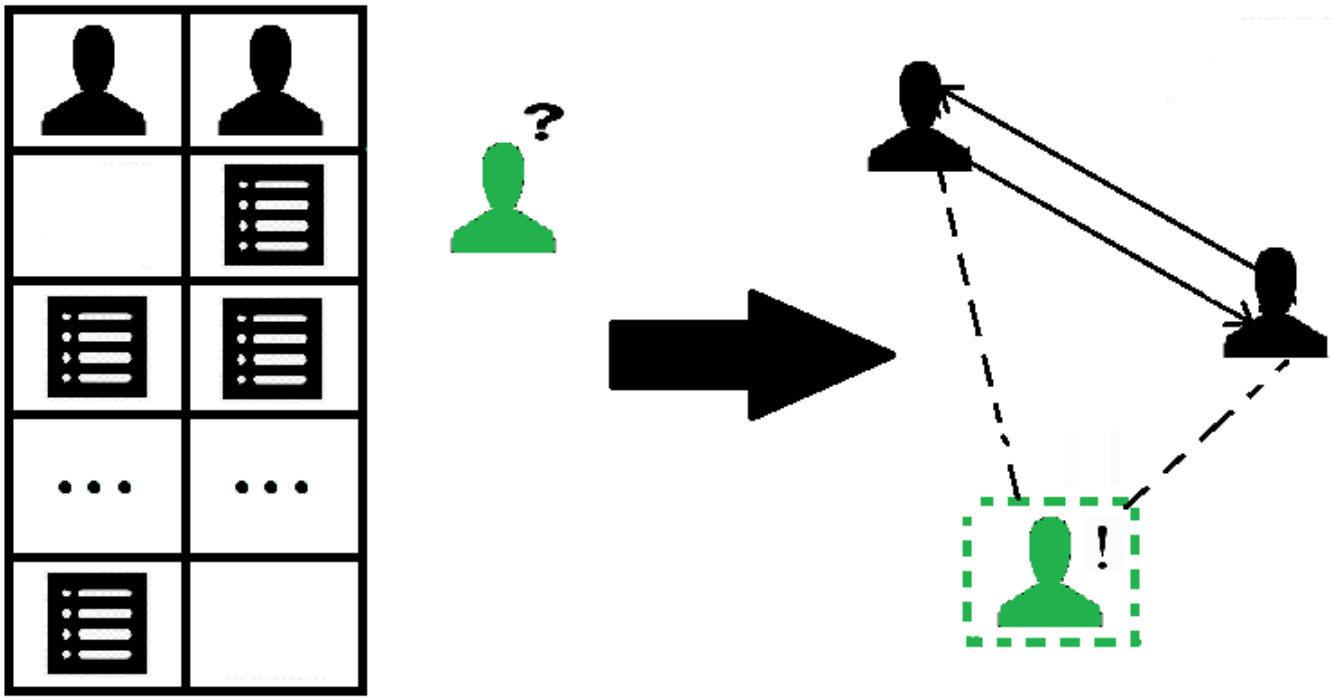


Figure 5 – From tabular representation to graph representation

Related Work

Graph Theory

1735

Leonhard Euler solved a popular puzzle about bridges

1980

(Buczak, JMJ): The Foundation of Homogeneous Graph Theory

2019

(Yanbang Cen et al.): Addresses the challenge of multiple types of nodes and edges in heterogeneous graphs

Related Work

Graph Transformations

2006

(Leskovec and Faloutsos): Graph sampling: Efficient method for preserving structure

2007

(Dhillon, Guan, and Kulis): Graph coarsening: Multilevel graph reduction without eigenvectors

2020

(Zheng, Huang, Jiang, and Li): Graph pruning: Removing all edges below the weight threshold

Related Work

Graph Embedding

2014

(Perozzi et al.): DeepWalk: Representation learning based on random walk and skip-gram model

2016

(Grover et al.): Node2Vec: Maximizes the likelihood of preserving network neighborhoods of nodes

2017

(Hamilton et al.): GraphSAGE: Sampling and aggregating information from neighboring nodes

2019

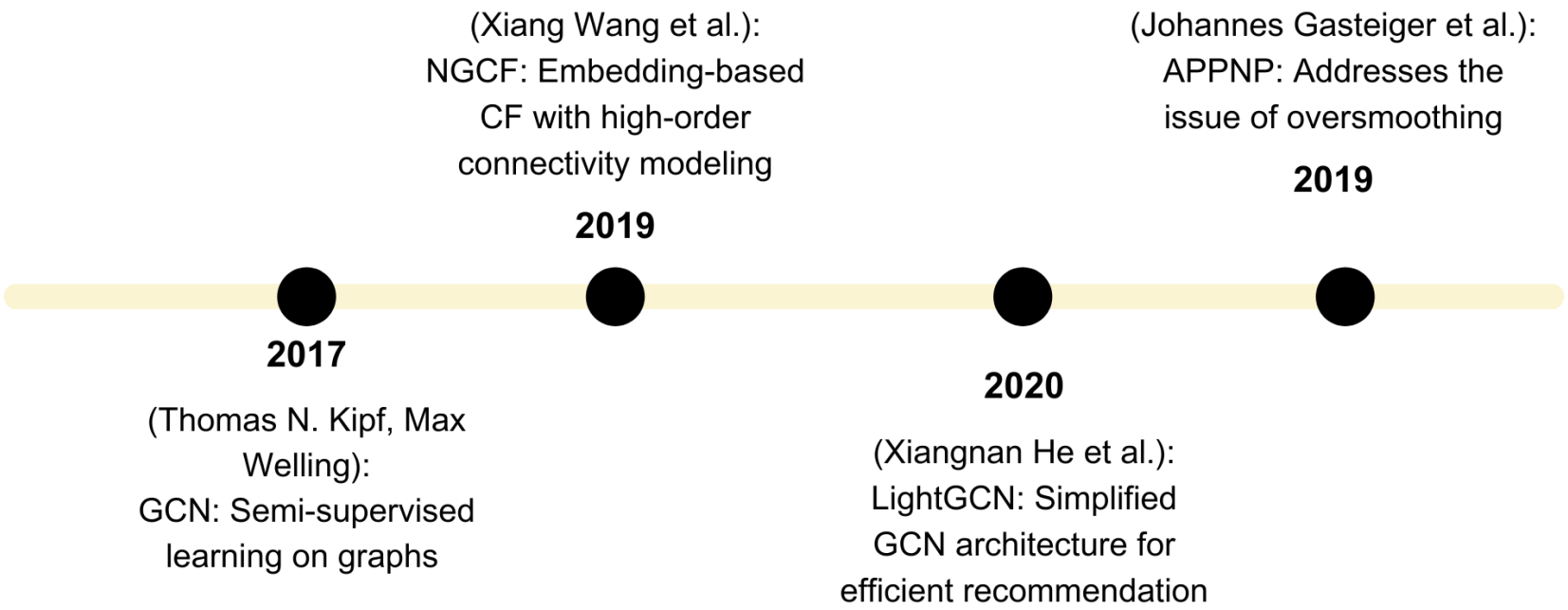
(Cen et al.): GATNE: Supports both transductive and inductive learning

2021

(Yang et al.): HGCN: Using information aggregation mechanism from different node and edge types

Related Work

Graph Convolution



Methodology

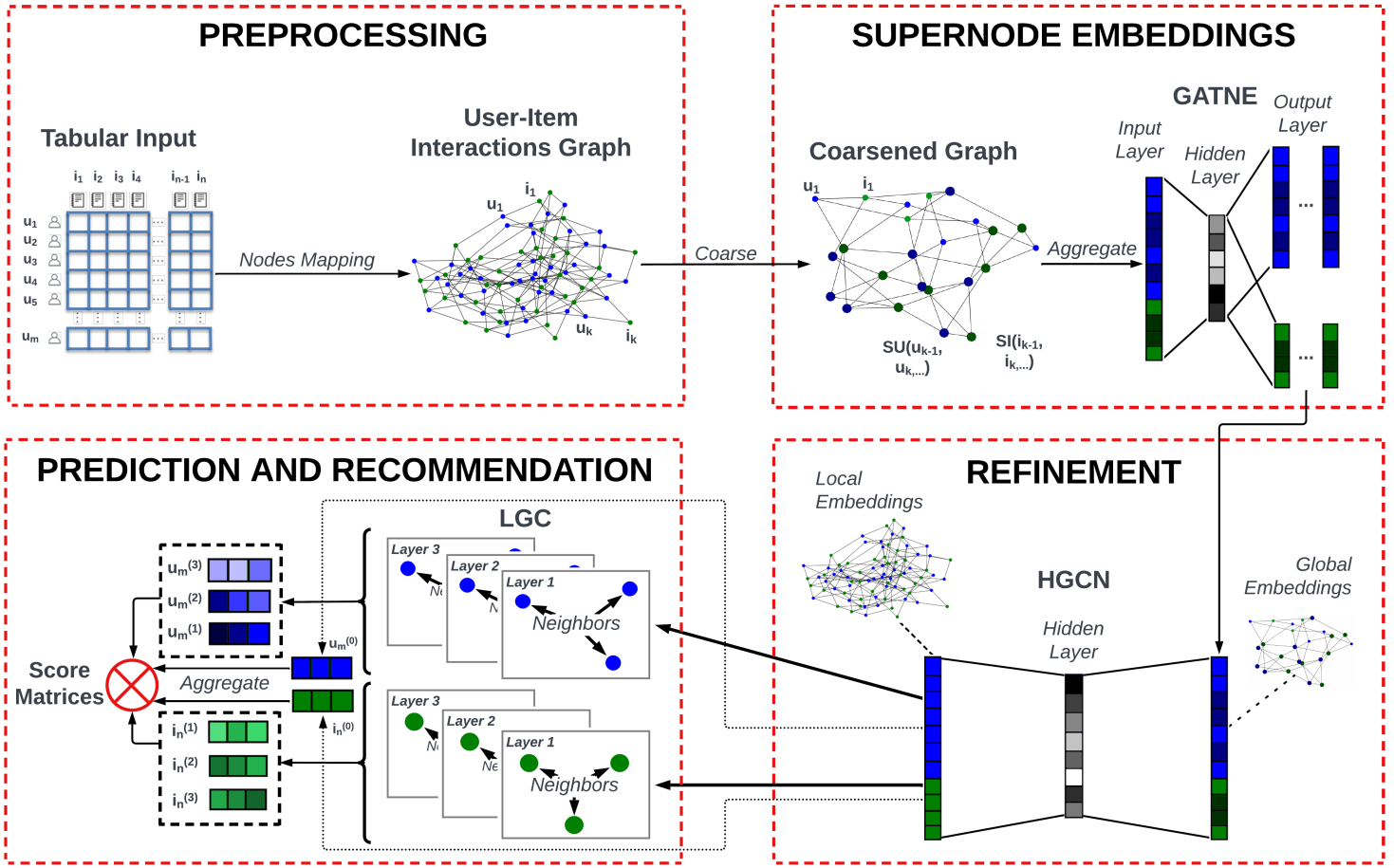


Figure 6 – He-LiGCN pipeline

Methodology

Preprocessing: Constructing Efficient Heterogeneous Graphs

- Iterative transformation from tabular input to heterogeneous graph

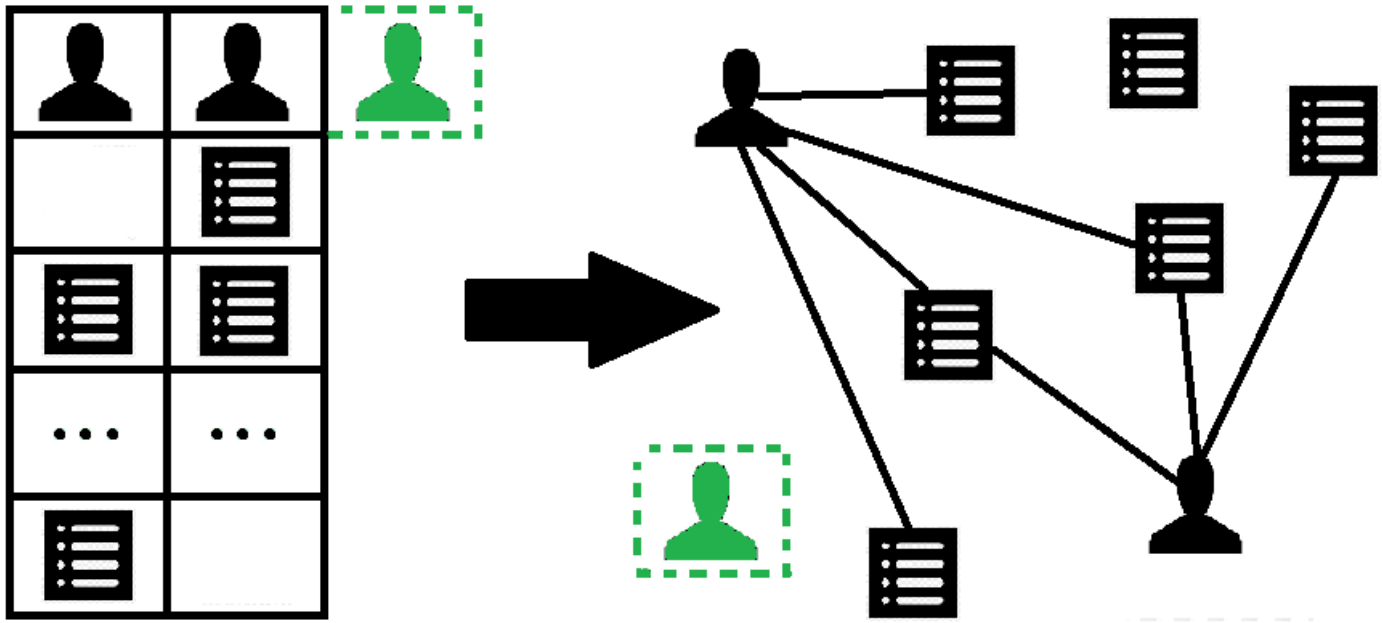


Figure 7 – Initialized heterogeneous graph process

Methodology

Preprocessing: Constructing Efficient Heterogeneous Graphs

- Identify and merge similar nodes in the graph coarsening process

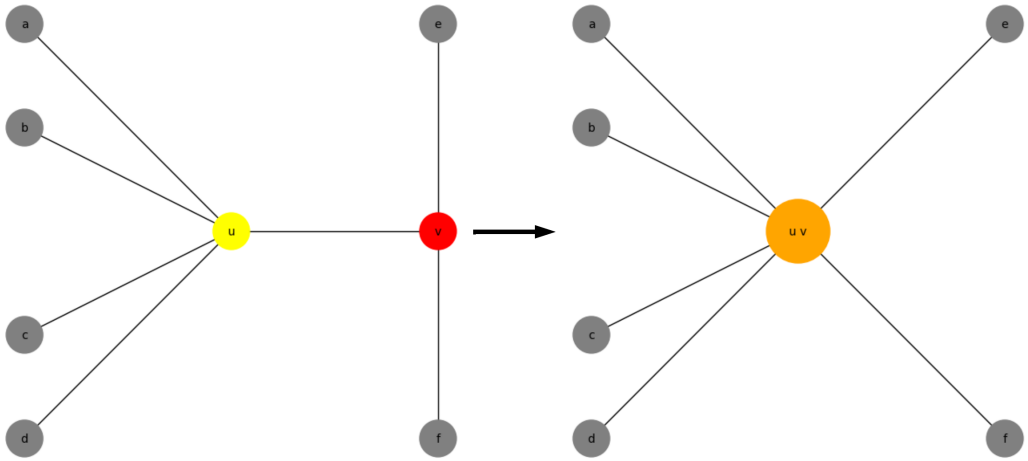


Figure 8 – Coarsening process

Jaccard Max

$$J(u,v) = \frac{|\mathcal{N}_u \cap \mathcal{N}_v|}{|\mathcal{N}_u \cup \mathcal{N}_v|}$$

Jaccard Weighted Random Sampling

$$p(u,v_j) = \frac{J(u,v_j)}{\sum_{i \in \mathcal{N}_u} J(u,v_j)}$$

Methodology

Preprocessing: Constructing Efficient Heterogeneous Graphs

- Group similar items into the same "bucket"
- Reducing computational complexity

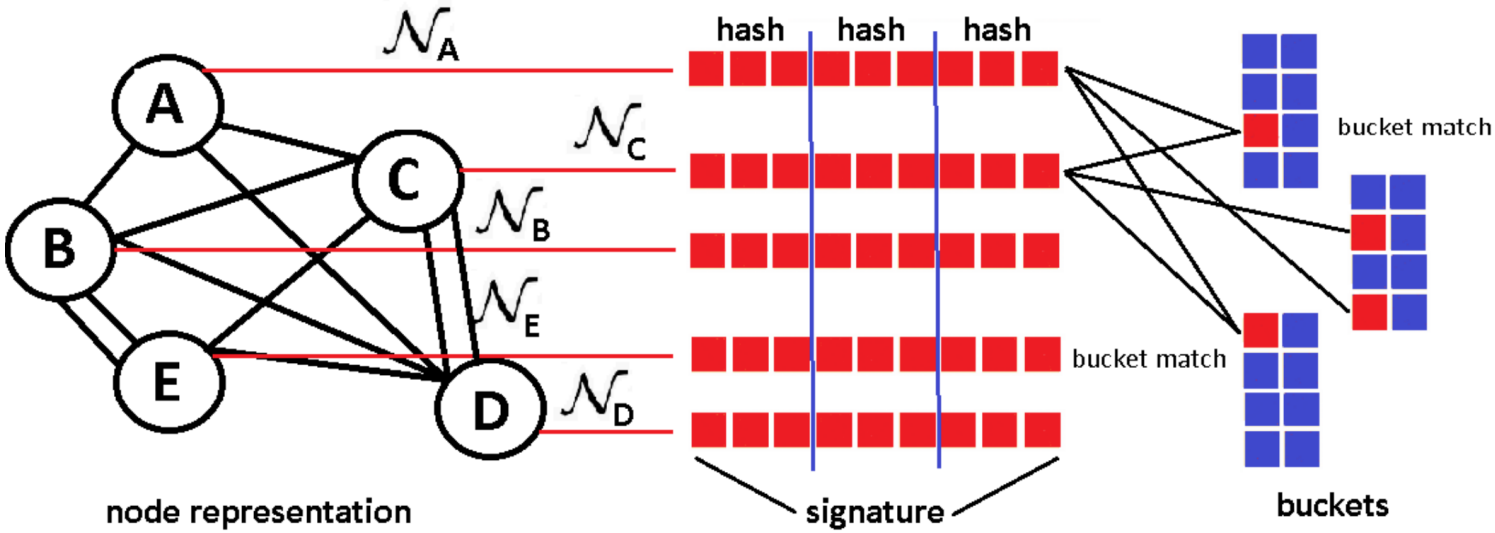


Figure 9 – Locality Sensitive Hashing (LSH) process

Locality Sensitive Hashing

$$P_{f_i \in F}[f_i(u) = f_i(v)] = P[\min \pi(u) = \min \pi(v)] \propto J(u, v)$$

Methodology

Supernode Embedding

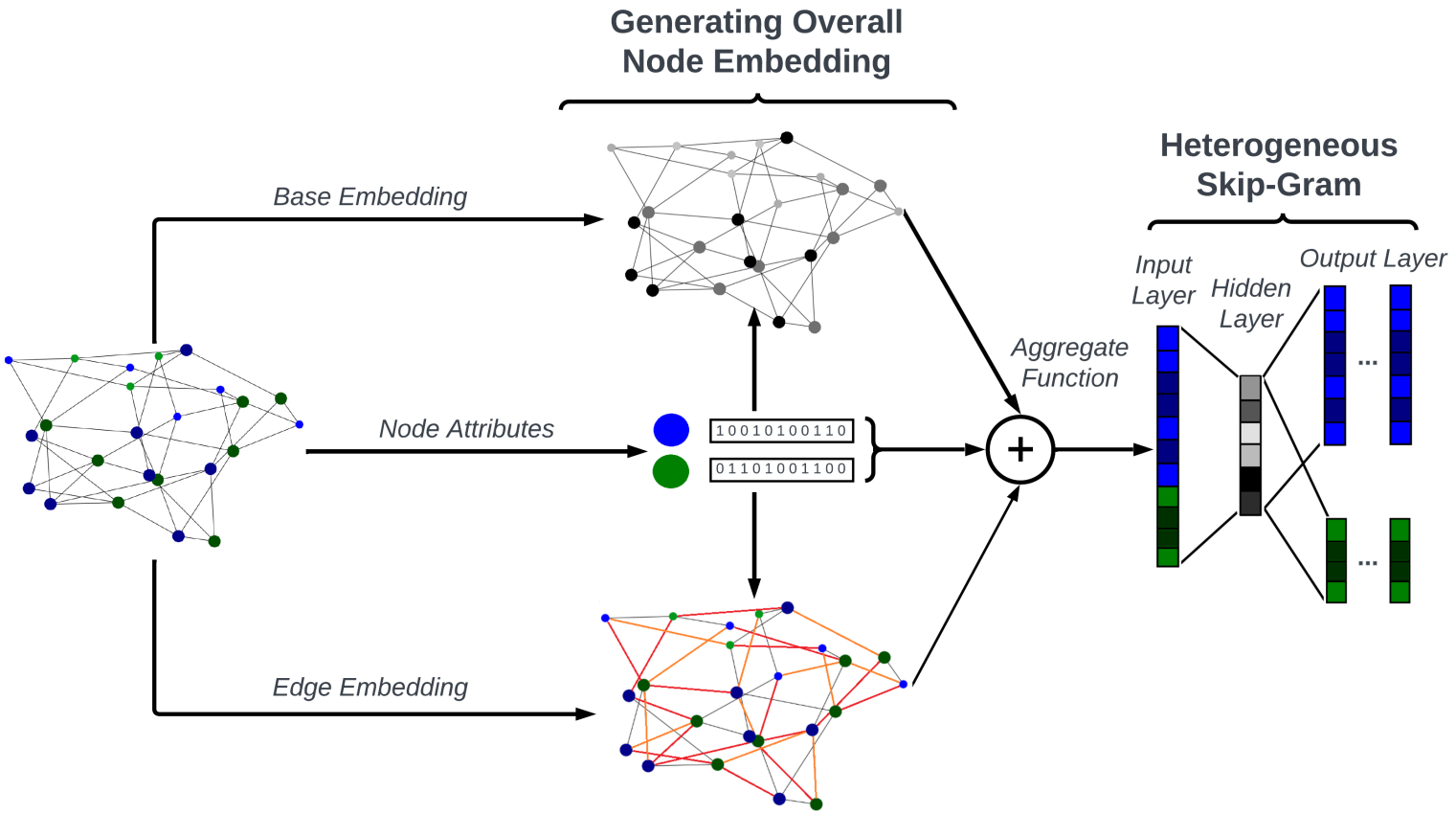


Figure 10 – The components of the GATNE model

Methodology

Supernode Embedding

- Generates initial embeddings for nodes in the coarsened graph
- Captures multiplex relationships and structural diversity

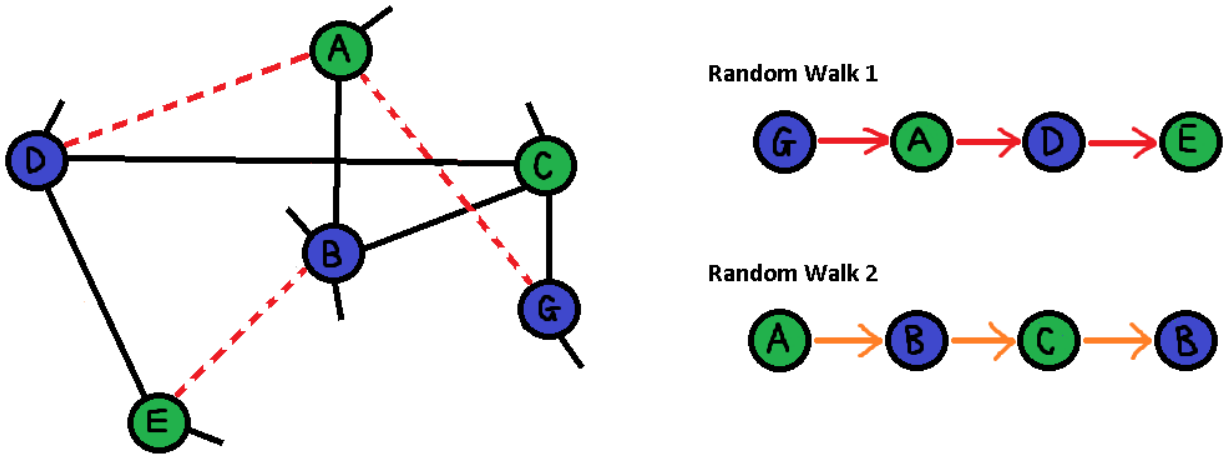


Figure 11 – Random walk implementation

Inductive Model: GATNE-I

$$\mathbf{v}_{i,r} = \mathbf{b}_i + \alpha_r \mathbf{M}_r^\top \mathbf{U}_i \mathbf{a}_{i,r} + \beta_r \mathbf{D}_z^\top \mathbf{x}_i$$

Methodology

Supernode Embedding

- Maximizes similarity for positive pairs
- Minimizes similarity for negative pairs

Embedding Optimization

$$\mathcal{L} = \sum_{v_j \in \mathcal{C}} -\log\left(\frac{e^{\mathbf{c}_j^\top \mathbf{v}_{i,r}}}{\sum_{k \in \mathcal{V}_t} e^{\mathbf{c}_k^\top \mathbf{v}_{i,r}}}\right) = -\log \sigma(\mathbf{c}_j^\top \mathbf{v}_{i,r}) - \sum_{l=1}^{L} \mathbb{E}_{v_k \sim P_t(v)} \left[\log \sigma(-\mathbf{c}_k^\top \mathbf{v}_{i,r})\right]$$

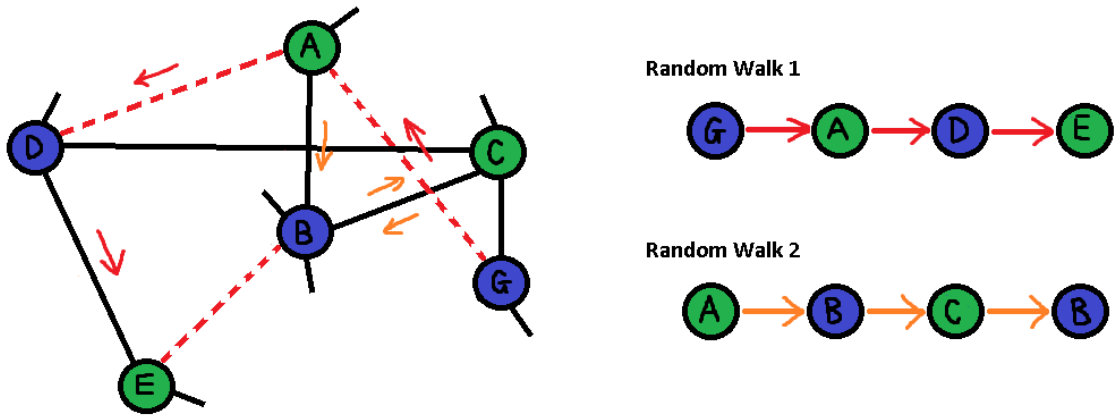


Figure 12 – Example of negative sample (random walk 1) and positive samples (random walk 2)

Methodology

Refinement

- Capturing both local and global structural information
- Reconstruct the embedding of the original graph $\mathcal{G}_0$ from $\mathcal{G}_m$

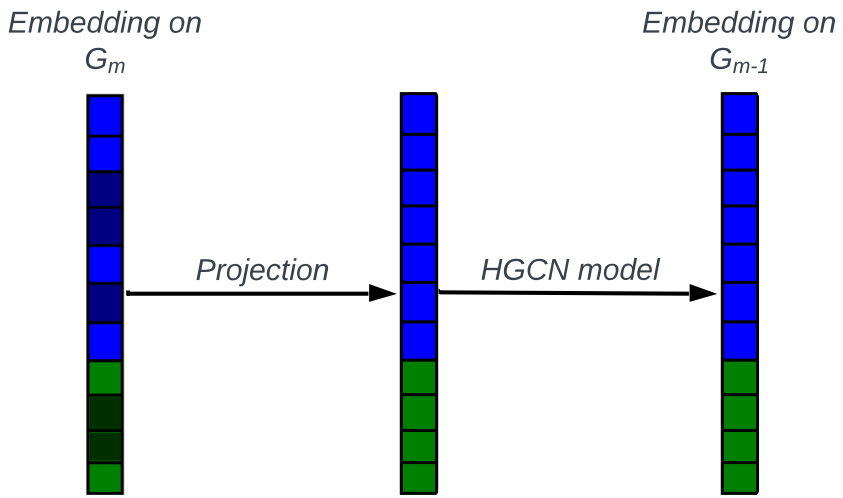


Figure 13 – Embedding generation using hierarchical projection and representation learning

Projection Equation

$$\mathbf{E}_i' = \mathbf{M}_{i,i+1}\mathbf{E}_{i+1}$$

Methodology

Refinement

- Refines embeddings by aggregating information from heterogeneous neighbors

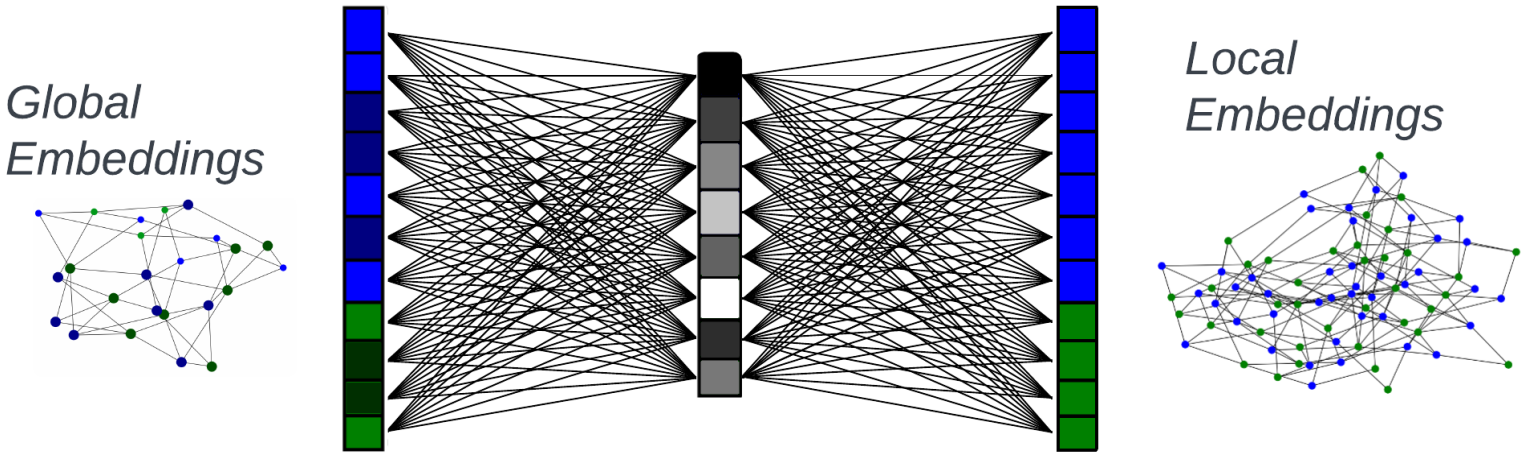


Figure 14 – From global embedding to local embedding

Refinement Learning

$$\mathbf{H}_i^{O_i'} = \text{LeakyReLU}\left(\mathbf{\Phi}_i^{\langle O_i \rangle} \cdot \mathbf{Z}_{i,:}^{\langle O_i \rangle} + \sum_{O_{\mathcal{N}} \in \mathcal{N}_{O_i}} \mathbf{\Phi}_i^{\langle O_{\mathcal{N}} \to O_i \rangle} \cdot \mathbf{Z}_{i,:}^{\langle O_{\mathcal{N}} \to O_i \rangle}\right)$$

Methodology

Refinement

- Refines embeddings by aggregating information from heterogeneous neighbors

Refinement Learning

$$\mathbf{H}_i^{O'_i} = \text{LeakyReLU}\left(\mathbf{\Phi}_i^{\langle O_i \rangle} \cdot \mathbf{Z}_{i,:}^{\langle O_i \rangle} + \sum_{O_{\mathcal{N}} \in \mathcal{N}_{O_i}} \mathbf{\Phi}_i^{\langle O_{\mathcal{N}} \to O_i \rangle} \cdot \mathbf{Z}_{i,:}^{\langle O_{\mathcal{N}} \to O_i \rangle}\right)$$

Explanations of Components

$$\mathbf{Z}^{\langle O_i \rangle} = \mathbf{H}^{O_i} \cdot \mathbf{W}^{\langle O_i \rangle}$$

$$\mathbf{Z}^{\langle O_{\mathcal{N}} \to O_i \rangle} = \hat{\mathbf{A}}^{\langle O_i - O_{\mathcal{N}} \rangle} \cdot \mathbf{H}^{O_{\mathcal{N}}} \cdot \mathbf{W}^{\langle O_{\mathcal{N}} \to O_i \rangle}$$

Methodology

Refinement

- Minimizes the discrepancy between predicted and ground-truth embeddings

Embedding Preservation

$$\mathcal{L} = \frac{1}{|\mathcal{V}_i|} \sum_{i=1}^{|\mathcal{V}_i|} \|\mathbf{E}_i - \mathbf{H}_i\|^2$$

Methodology

Prediction and Recommendation

- Combines embeddings from multiple layers to capture user-item interactions
- Generating recommendation score matrices

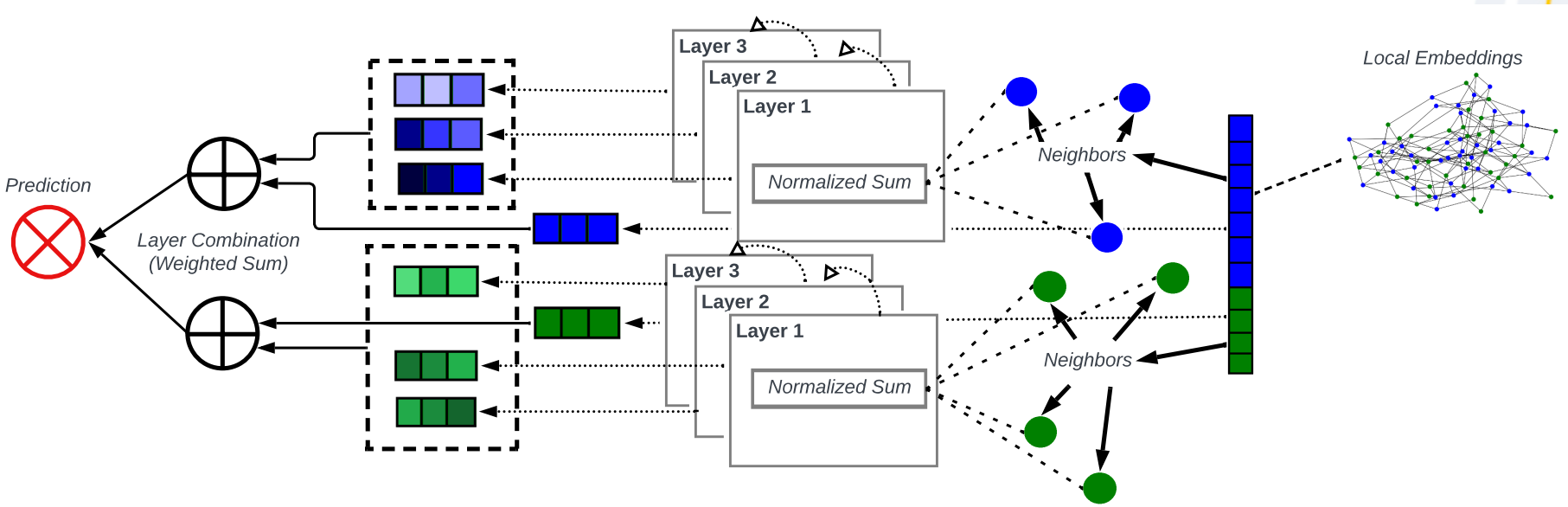


Figure 15 – Light Graph Convolution

Aggregation Formula

$$\mathbf{e}_u^{(k+1)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u||\mathcal{N}_i|}} \mathbf{e}_i^{(k)} \quad ; \quad \mathbf{e}_i^{(k+1)} = \sum_{u \in \mathcal{N}_i} \frac{1}{\sqrt{|\mathcal{N}_u||\mathcal{N}_i|}} \mathbf{e}_u^{(k)}$$

Methodology

Prediction and Recommendation

- Combines embeddings from multiple layers to capture user-item interactions
- Generates recommendation scores for user-item pairs

Layer Combination

$$\mathbf{e}_u = \sum_{k=0}^{K} \frac{1}{K+1} \mathbf{e}_u^{(k)}$$

$$\mathbf{e}_i = \sum_{k=0}^{K} \frac{1}{K+1} \mathbf{e}_i^{(k)}$$

Model Prediction

$$\hat{y}_{ui} = \mathbf{e}_u^\top \mathbf{e}_i$$

Methodology

Prediction and Recommendation

- Represents user-item interactions in a matrix form

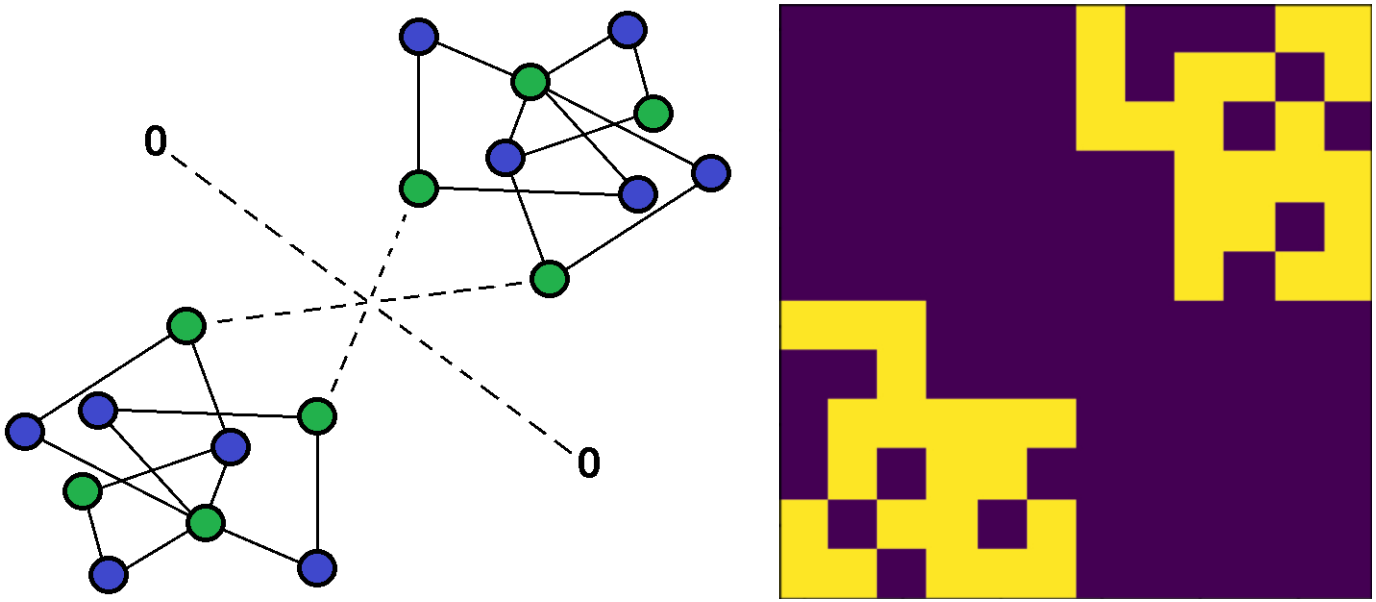


Figure 16 – Adjacency matrix representation

Normalized Adjacency Matrix Calculation

$$\tilde{\mathbf{A}} = \mathbf{D}^{-\frac{1}{2}} \begin{pmatrix} \mathbf{0} & \mathbf{R} \\ \mathbf{R}^\top & \mathbf{0} \end{pmatrix} \mathbf{D}^{-\frac{1}{2}}$$

Methodology

Prediction and Recommendation

- Mitigates over-smoothing by preserving node distinctiveness
- Maintains node distinctiveness while allowing deep architectures

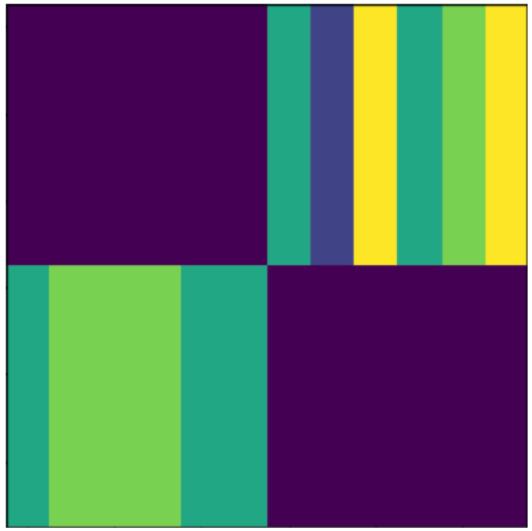


Figure 17 – Oversmoothing problem in GCN

Efficient Layer Combination

$$\mathbf{E} = \beta\mathbf{E}^{(0)} + \sum_{k=0}^{K} \beta(1-\beta)^k \tilde{\mathbf{A}}^k \mathbf{E}^{(0)} + (1-\beta)^K \tilde{\mathbf{A}}^K \mathbf{E}^{(0)}$$

Methodology

Prediction and Recommendation

- Ranks items that user have interacted with higher than items they haven't

Bayesian Personalized Ranking

$$\mathcal{L}_{\text{BPR}} = -\sum_{u=1}^{M} \sum_{i \in \mathcal{N}_u} \sum_{j \notin \mathcal{N}_u} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda \|\mathbf{\Theta}\|^2$$

Methodology
Evaluation Metrics

- Measures retrieval performance

Recall@k

$$\text{Recall@k} = \frac{\text{Number of relevant items retrieved in top } k}{\text{Total number of relevant items}}$$

- Measures ranking quality, penalizing incorrect placement of relevant items

NDCG@10

$$\text{NDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}} = \frac{\sum_{i=1}^{k} \frac{\text{rel}_i}{\log_2(i+1)}}{\sum_{i=1}^{k} \frac{\text{rel}_i^{\text{ideal}}}{\log_2(i+1)}}$$

Experiment and Results

- Performance comparison at different layer depths

Table 1 – Scalability and efficiency across graph layers

Parameter	He-LiGCN	NGCF	LightGCN
Embedding Size	64	64	64
Learning Rate	0.001	0.001	0.001
Batch Size	1024	1024	1024
Number of Layers	3	3	3
Coarsening Levels	2	N/A	N/A
Regularization Coefficient	10^{-2}	10^{-2}	10^{-2}
Optimizer	Adam	Adam	Adam

Experiment and Results

- The training performance over 200 epochs

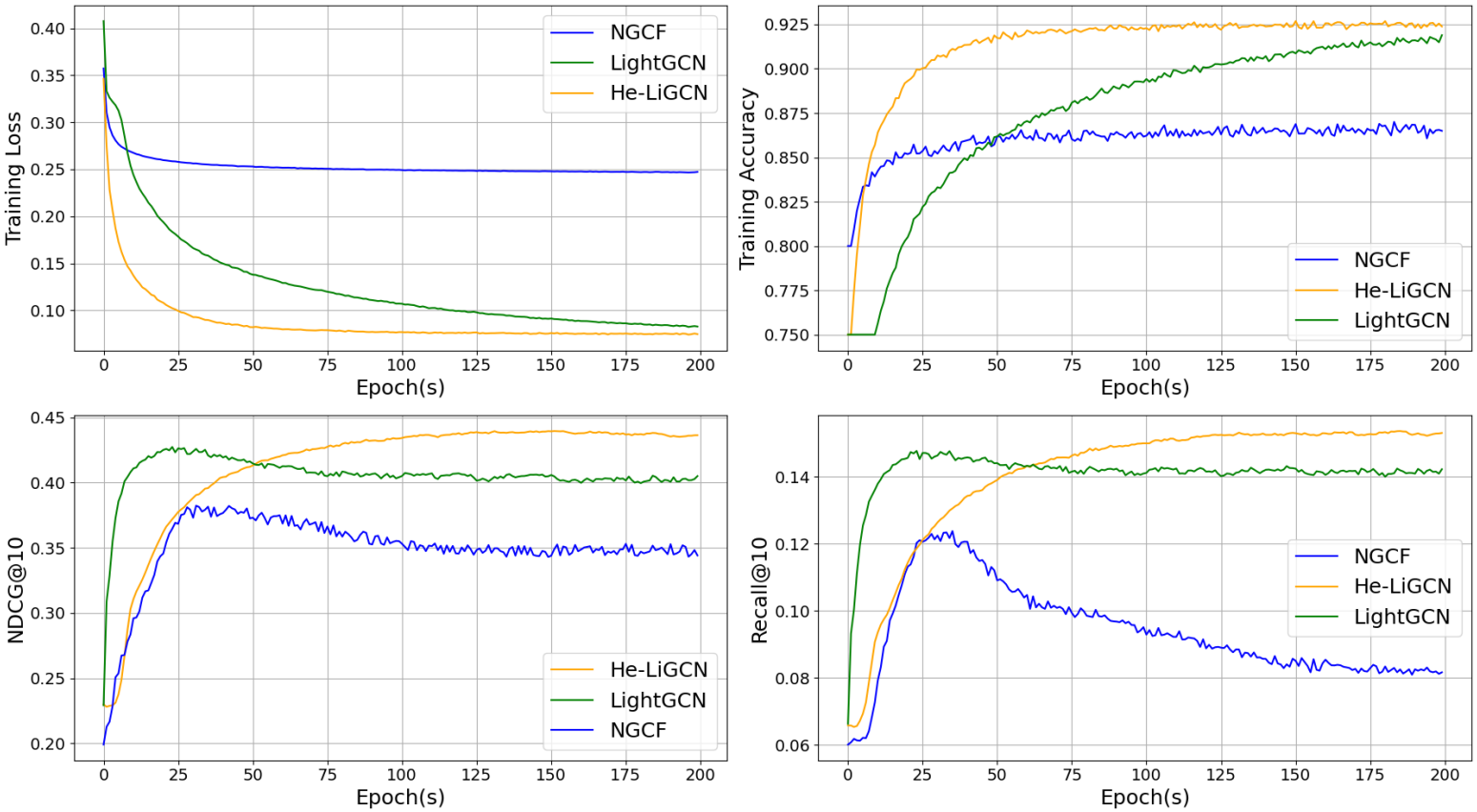


Figure 18 – Comparative analysis of recommendation models

Experiment and Results

- Performance comparison at different layer depths

Table 2 – Scalability and efficiency across graph layers

# Layers	Model	NDCG@10	Recall@10
1	NGCF	0.379	0.120
	LightGCN	0.423	0.140
	He-LiGCN	0.444	0.155
2	NGCF	0.382	0.123
	LightGCN	0.420	0.147
	He-LiGCN	0.446	0.156
3	NGCF	0.377	0.126
	LightGCN	0.429	0.148
	He-LiGCN	**0.448**	**0.157**
4	NGCF	0.370	0.124
	LightGCN	0.428	0.147
	He-LiGCN	0.447	0.150

Experiment and Results

- Ablation studies
  - 1: graph coarsening
  - 2: GATNE and HGCN embeddings
  - 3: LGC aggregation

Table 3 – Ablation study results with removed components

Model Variant	Recall@10	NDCG@10
He-LiGCN-[1, 2]	0.148	0.435
He-LiGCN-[1, 3]	0.141	0.420
He-LiGCN-[2, 3]	0.152	0.442
Complete He-LiGCN	**0.157**	**0.448**

- Effectiveness studies

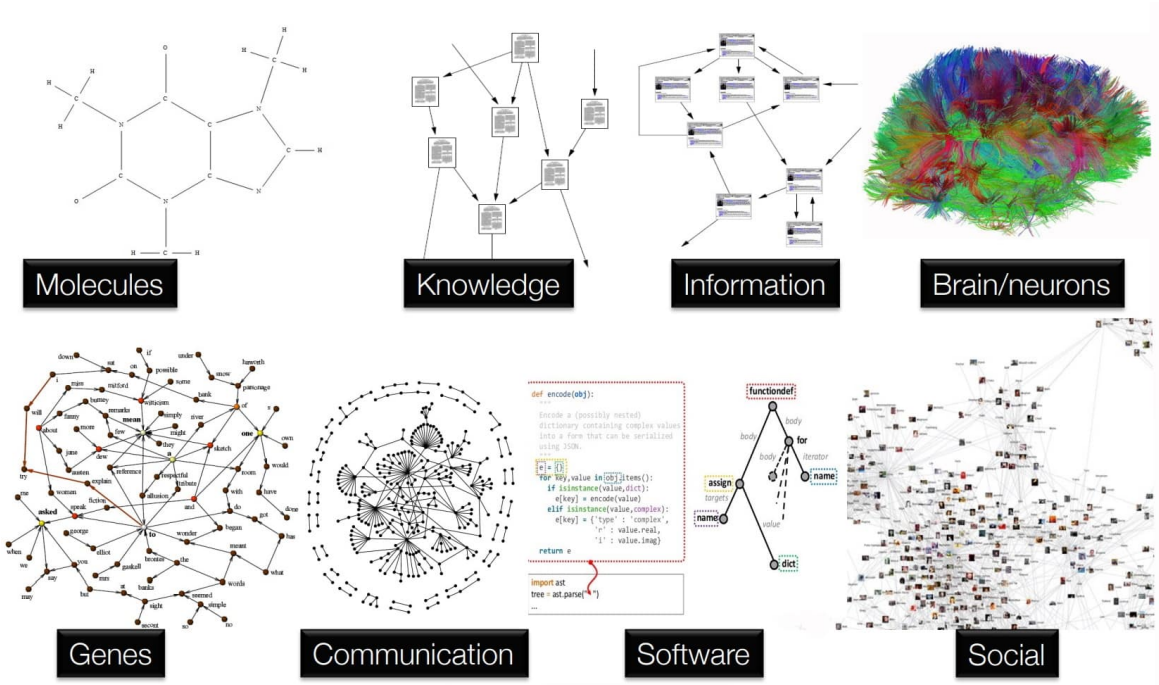
Table 4 – Average computational cost per epoch

Model	training time (seconds)
He-LiGCN	10.2
NGCF	12.5
LightGCN	**9.8**

Conclusions

Key Contributions

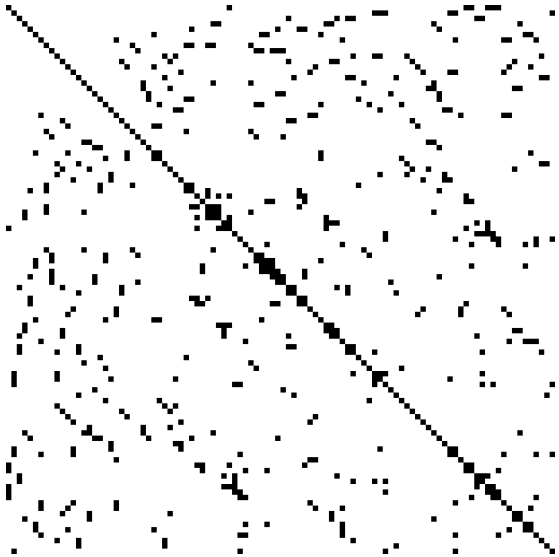
- Addresses scalability and computational challenges of traditional GNNs
- Handle large, complex, and heterogeneous user-item interaction graphs efficiently
- Provides a practical and scalable solution for next-generation personalized recommendation systems
- Enhances embedding quality, balances accuracy and computational efficiency



Conclusions

Limitations

- May struggle with very sparse data or highly dynamic graph structures
- Effectiveness of graph coarsening may vary with different datasets



Conclusions

Future Work

- Explore dynamic graph structures and real-time data integration
- Consider incorporating additional side information for improved personalization

